DATABASE SYSTEMS / PROJECT PRESENTATION

FASHION SHOW MANAGEMENT SYSTEM

A Java web application using JDBC and MySQL

BY

Swayamshree Saswatee Sahu

Reg. No.: 25BAI0210

BACSE202 | Database Systems

PROJECT MODULES

01 **Models**

02 Designers

03 Fashion Shows

04 Outfits

BACSE202

CREATE — Adding New Records

The CREATE operation inserts a new record into the selected project table.

How the feature works

1. Open the required module (Models, Designers, Fashion Shows or Outfits).
2. Select Add New and enter the required field values.
3. The application validates the form data.
4. JDBC sends an INSERT statement to MySQL.
5. The new record appears in the listing table.

SQL EXAMPLE — INSERT A DESIGNER

```
INSERT INTO DESIGNERS
  (DESIGNER_ID, DESIGNER_NAME, BRAND_NAME, CONTACT, EMAIL)
VALUES
  (12, 'Meera Rao', 'Meera Studio', '9876543230',
   'meera@gmail.com');
```

JDBC IMPLEMENTATION

```
PreparedStatement ps = con.prepareStatement(sql);
ps.setInt(1, designerId);
ps.setString(2, designerName);
ps.executeUpdate();
```

The exact column names should match the table definition used by the application.

READ — Retrieving Records

READ displays stored information from the database in the website's tables.

READ ALL

Display all model records.

```
SELECT * FROM MODELS
ORDER BY MODEL_ID;
```

READ ONE

Find a designer using its primary key.

```
SELECT * FROM DESIGNERS
WHERE DESIGNER_ID = 1;
```

READ RELATED

Display outfits belonging to a designer.

```
SELECT OUTFIT_NAME, CATEGORY
FROM OUTFITS
WHERE DESIGNER_ID = 1;
```

JDBC FLOW: Connection → PreparedStatement → executeQuery() → ResultSet → HTML table

UPDATE — Editing Existing Records

UPDATE changes selected values while keeping the same record identifier.

Update process

01 User selects Edit for a particular row.

02 The application loads existing values into a form.

03 User modifies the required fields.

04 Java validates the updated values.

05 JDBC executes UPDATE using the record's primary key.

SQL EXAMPLE — UPDATE A DESIGNER

```
UPDATE DESIGNERS
SET BRAND_NAME = 'Meera Atelier'
WHERE DESIGNER_ID = 1;
```

IMPORTANT

The WHERE clause identifies the record to update. Without it, every row in the table could be changed.

DELETE — Removing Records

DELETE removes a selected record from a table.

Safe delete workflow

1. Select Delete for the required record.
2. Confirm the action on the website.
3. The application sends a DELETE request.
4. MySQL removes the row identified by its ID.
5. The listing refreshes to show the remaining records.

SQL EXAMPLE — DELETE AN OUTFIT

```
DELETE FROM OUTFITS
WHERE OUTFIT_ID = 5;
```

REFERENTIAL INTEGRITY

A foreign-key constraint may prevent deletion when another record depends on the selected row. This protects related data.

CRUD Across the Four Project Modules

The same four operations are available throughout the application.

MODELS

Model name, gender, contact, height and experience

AVAILABLE OPERATIONS

CREATE | READ | UPDATE | DELETE

DESIGNERS

Designer name, brand, contact and email

AVAILABLE OPERATIONS

CREATE | READ | UPDATE | DELETE

FASHION SHOWS

Title, date, venue, theme and status

AVAILABLE OPERATIONS

CREATE | READ | UPDATE | DELETE

OUTFITS

Category, material, size, colour, availability and IDs

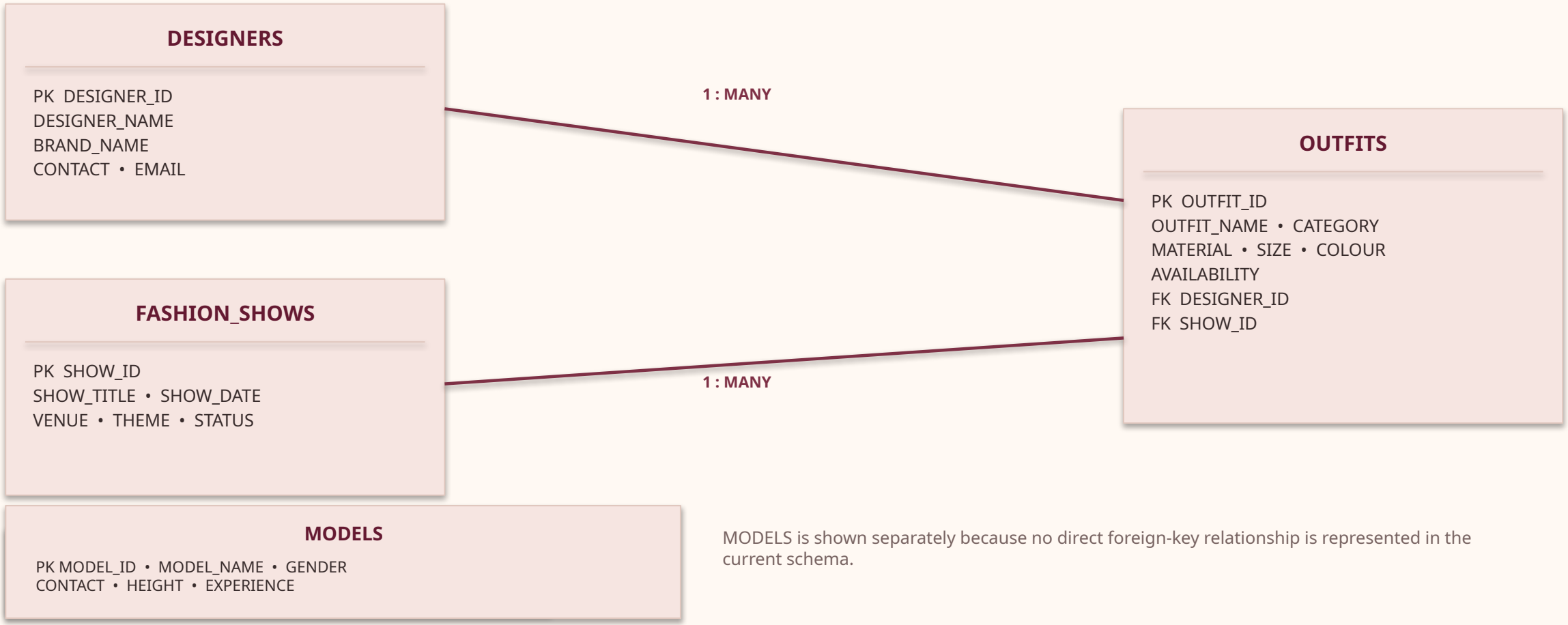
AVAILABLE OPERATIONS

CREATE | READ | UPDATE | DELETE

Primary keys identify records. In OUTFITS, Designer ID and Fashion Show ID link each outfit to its related records.

Entity–Relationship Diagram

The OUTFITS table stores foreign keys to DESIGNERS and FASHION_SHOWS.



MODELS is shown separately because no direct foreign-key relationship is represented in the current schema.

Normalisation Technique

Normalisation organises data to reduce unnecessary duplication and update anomalies.

1NF — **FIRST NORMAL FORM**

Each field stores one value. Repeating groups are avoided; for example, a designer's contact and email are stored as separate attributes.

2NF — **SECOND NORMAL FORM**

The design uses single-column primary keys for the listed entities, so partial dependency on a composite key is not expected.

3NF — **THIRD NORMAL FORM**

Designer and fashion-show details are stored in their own tables. OUTFITS stores their IDs instead of repeating all parent details.

These are design-level observations; confirm the actual table constraints before claiming formal normalisation.

DDL, DML and TCL — Five Examples Each

Examples are written for the project tables; run them in a test database before demonstration.

DDL / STRUCTURE

```
1  CREATE TABLE SAMPLE_LOG (
     LOG_ID INT PRIMARY KEY,
     NOTE VARCHAR(100)
   );

2  ALTER TABLE MODELS
   ADD COLUMN CITY VARCHAR(50);

3  CREATE TABLE DESIGNER_BACKUP
   AS SELECT * FROM DESIGNERS;

4  TRUNCATE TABLE SAMPLE_LOG;

5  DROP TABLE DESIGNER_BACKUP;
```

DML / DATA

```
1  INSERT INTO DESIGNERS
   (DESIGNER_ID, DESIGNER_NAME,
    BRAND_NAME, CONTACT, EMAIL)
   VALUES (12,'Meera Rao','Meera Studio',
   '9876543230','meera@gmail.com');

2  UPDATE DESIGNERS
   SET BRAND_NAME='Meera Atelier'
   WHERE DESIGNER_ID=12;

3  DELETE FROM DESIGNERS
   WHERE DESIGNER_ID=12;

4  INSERT INTO OUTFITS
   (OUTFIT_ID, OUTFIT_NAME, DESIGNER_ID, SHOW_ID)
   VALUES (15,'Rose Gown',1,1);

5  UPDATE OUTFITS
   SET AVAILABILITY='Reserved'
   WHERE OUTFIT_ID=15;
```

TCL / TRANSACTION

```
1  START TRANSACTION;

2  SAVEPOINT before_update;

3  UPDATE OUTFITS
   SET AVAILABILITY='Reserved'
   WHERE OUTFIT_ID=15;

4  ROLLBACK TO SAVEPOINT before_update;

5  COMMIT;
   -- or use ROLLBACK to undo
   -- uncommitted changes.
```

Arithmetic, Logical & Comparison Operators

Operators calculate values and combine conditions when retrieving project data.

ARITHMETIC

+ - * / MOD

```
SELECT MODEL_NAME, HEIGHT_CM,
       HEIGHT_CM + 5 AS NEW_HEIGHT
FROM MODELS;
```

Adds five centimetres to each model's height.

COMPARISON

= <> > < >= <= LIKE

```
SELECT MODEL_NAME, EXPERIENCE_YEARS
FROM MODELS
WHERE EXPERIENCE_YEARS >= 2;
```

Returns models with at least two years of experience.

LOGICAL

AND OR NOT

```
SELECT SHOW_TITLE, SHOW_DATE
FROM FASHION_SHOWS
WHERE SHOW_DATE >= '2026-01-01'
  AND STATUS <> 'Cancelled';
```

Combines two conditions to filter shows.

Arithmetic changes numeric values; comparison evaluates a condition; logical operators combine conditions.

Subqueries — Five Statements with Outputs

Each example includes the SQL statement and an illustrative result based on sample records.

01 ABOVE-AVERAGE HEIGHT

```
SELECT MODEL_NAME, HEIGHT_CM FROM MODELS
WHERE HEIGHT_CM > (SELECT AVG(HEIGHT_CM) FROM MODELS);
```

Illustrative output: A model with height 182 cm (if above the stored average).

02 OUTFITS BY DESIGNER

```
SELECT OUTFIT_NAME FROM OUTFITS
WHERE DESIGNER_ID = (SELECT DESIGNER_ID FROM DESIGNERS
WHERE DESIGNER_NAME='Meera Rao');
```

Illustrative output: Rose Gown.

03 LATEST FASHION SHOW

```
SELECT SHOW_TITLE, SHOW_DATE FROM FASHION_SHOWS
WHERE SHOW_DATE = (SELECT MAX(SHOW_DATE) FROM FASHION_SHOWS);
```

Illustrative output: the show with the maximum stored date.

04 DESIGNERS WITH OUTFITS

```
SELECT DESIGNER_NAME FROM DESIGNERS d
WHERE EXISTS (SELECT 1 FROM OUTFITS o
WHERE o.DESIGNER_ID=d.DESIGNER_ID);
```

Illustrative output: designers referenced by at least one outfit.

05 OUTFITS ABOVE AVERAGE ID

```
SELECT OUTFIT_NAME, OUTFIT_ID FROM OUTFITS
WHERE OUTFIT_ID > (SELECT AVG(OUTFIT_ID) FROM OUTFITS);
```

Illustrative output: outfit rows whose ID is greater than the average ID.

Outputs are illustrative; the exact rows depend on the records currently stored in MySQL.

Joins — All Common Join Types

Joins combine related rows using matching columns such as DESIGNER_ID and SHOW_ID.

INNER JOIN

Only rows with a match in both tables.

```
SELECT o.OUTFIT_NAME, d.DESIGNER_NAME
FROM OUTFITS o INNER JOIN DESIGNERS d
ON o.DESIGNER_ID=d.DESIGNER_ID;
```

LEFT OUTER JOIN

All designers, plus matching outfits.

```
SELECT d.DESIGNER_NAME, o.OUTFIT_NAME
FROM DESIGNERS d LEFT JOIN OUTFITS o
ON d.DESIGNER_ID=o.DESIGNER_ID;
```

RIGHT OUTER JOIN

All outfits, plus matching designers.

```
SELECT d.DESIGNER_NAME, o.OUTFIT_NAME
FROM DESIGNERS d RIGHT JOIN OUTFITS o
ON d.DESIGNER_ID=o.DESIGNER_ID;
```

FULL OUTER JOIN

All matched and unmatched rows.

```
MySQL note: emulate FULL OUTER JOIN with
LEFT JOIN UNION RIGHT JOIN.
```

CROSS JOIN

Every possible pair of rows.

```
SELECT d.DESIGNER_NAME, s.SHOW_TITLE
FROM DESIGNERS d CROSS JOIN FASHION_SHOWS s;
```

SELF JOIN

A table joined to itself.

```
Example requires a self-referencing column;
none is shown in the current project schema.
```

Control Structures

These Oracle PL/SQL examples demonstrate selection and repetition for the database course.

SELECTION — IF / ELSIF / ELSE

```
DECLARE
  v_experience NUMBER := 4;
  v_level VARCHAR2(20);
BEGIN
  IF v_experience = 0 THEN
    v_level := 'New';
  ELSIF v_experience < 5 THEN
    v_level := 'Developing';
  ELSE
    v_level := 'Experienced';
  END IF;
  DBMS_OUTPUT.PUT_LINE(v_level);
END;
/
```

ITERATION — FOR LOOP

```
BEGIN
  FOR i IN 1..3 LOOP
    DBMS_OUTPUT.PUT_LINE('Show ' || i);
  END LOOP;
END;
/
```

A conditional structure selects a path; a loop repeats statements while its control condition permits.

Stored Procedures

A stored procedure is a named PL/SQL program unit saved in the database for reuse.

```
CREATE OR REPLACE PROCEDURE show_designer_outfits (
  p_designer_id IN NUMBER
) AS
BEGIN
  FOR r IN (
    SELECT OUTFIT_NAME, CATEGORY
    FROM OUTFITS
    WHERE DESIGNER_ID = p_designer_id
  ) LOOP
    DBMS_OUTPUT.PUT_LINE(
      r.OUTFIT_NAME || ' - ' || r.CATEGORY
    );
  END LOOP;
END;
/

BEGIN
  show_designer_outfits(1);
END;
/
```

WHY PROCEDURES?

01 Reusable database logic

02 Centralised operations

03 Accept input parameters

04 Can control permissions

Course PL/SQL example; separate from the MySQL web backend.

Cursors

A cursor processes the rows returned by a query, one row at a time.

EXPLICIT CURSOR STEPS

01 DECLARE

02 OPEN

03 FETCH

04 CHECK %NOTFOUND

05 CLOSE

Each FETCH retrieves the next row.

EXPLICIT CURSOR EXAMPLE

```
DECLARE
  CURSOR c_shows IS
    SELECT SHOW_ID, SHOW_TITLE, SHOW_DATE
    FROM FASHION_SHOWS
    ORDER BY SHOW_DATE;
  v_id FASHION_SHOWS.SHOW_ID%TYPE;
  v_title FASHION_SHOWS.SHOW_TITLE%TYPE;
  v_date FASHION_SHOWS.SHOW_DATE%TYPE;
BEGIN
  OPEN c_shows;
  LOOP
    FETCH c_shows INTO v_id, v_title, v_date;
    EXIT WHEN c_shows%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(v_title);
  END LOOP;
  CLOSE c_shows;
END;
/
```

Views

A view is a virtual table based on a stored SELECT statement.

A view simplifies repeated queries and can present selected information.

WHY USE A VIEW?

01 Reuse a query

02 Limit displayed columns

03 Simplify related-table reports

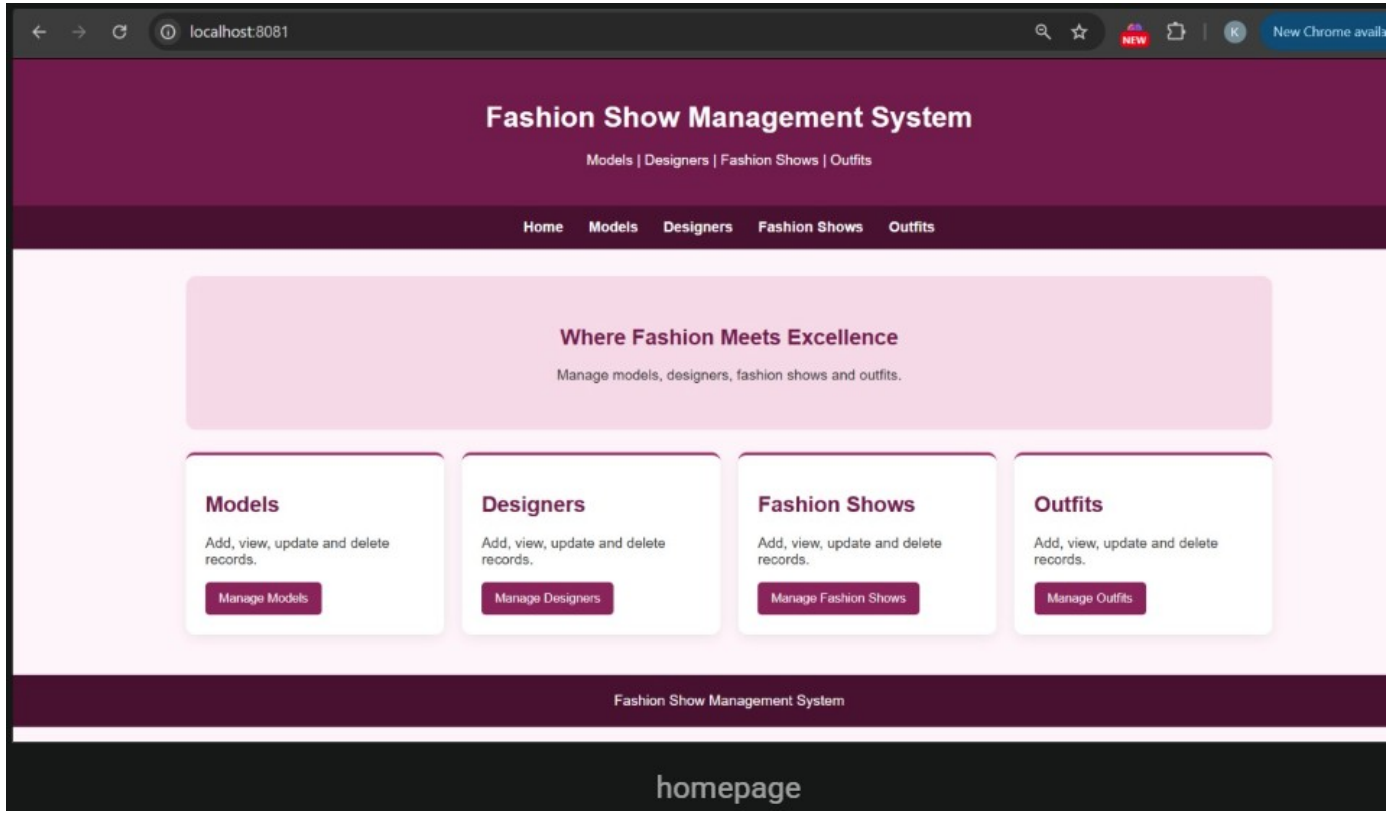
CREATE A VIEW

```
CREATE OR REPLACE VIEW OUTFIT_DESIGNER_VIEW AS
SELECT o.OUTFIT_ID, o.OUTFIT_NAME, o.CATEGORY,
       d.DESIGNER_NAME, d.BRAND_NAME
FROM OUTFITS o
JOIN DESIGNERS d
  ON o.DESIGNER_ID=d.DESIGNER_ID;
```

QUERY THE VIEW

```
SELECT * FROM OUTFIT_DESIGNER_VIEW
ORDER BY OUTFIT_ID;
```

01 Homepage — Main Dashboard



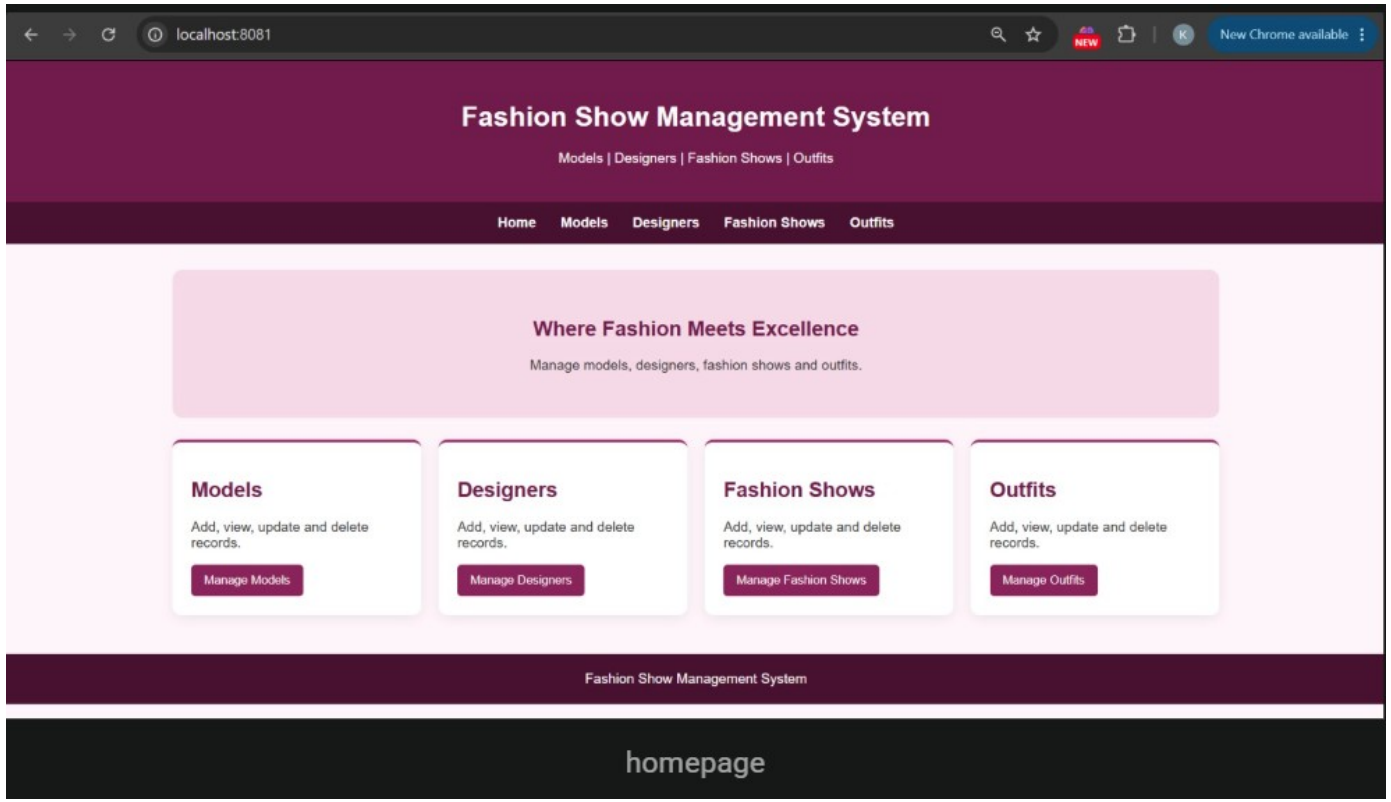
WHAT THIS SCREEN SHOWS

The homepage is the main entry point of the system. The navigation bar links to Models, Designers, Fashion Shows and Outfits. The four cards provide shortcuts to manage each module.

DBMS / PROJECT EXPLANATION

The dashboard gives users one place to access all four parts of the application.

02 Homepage — Navigation



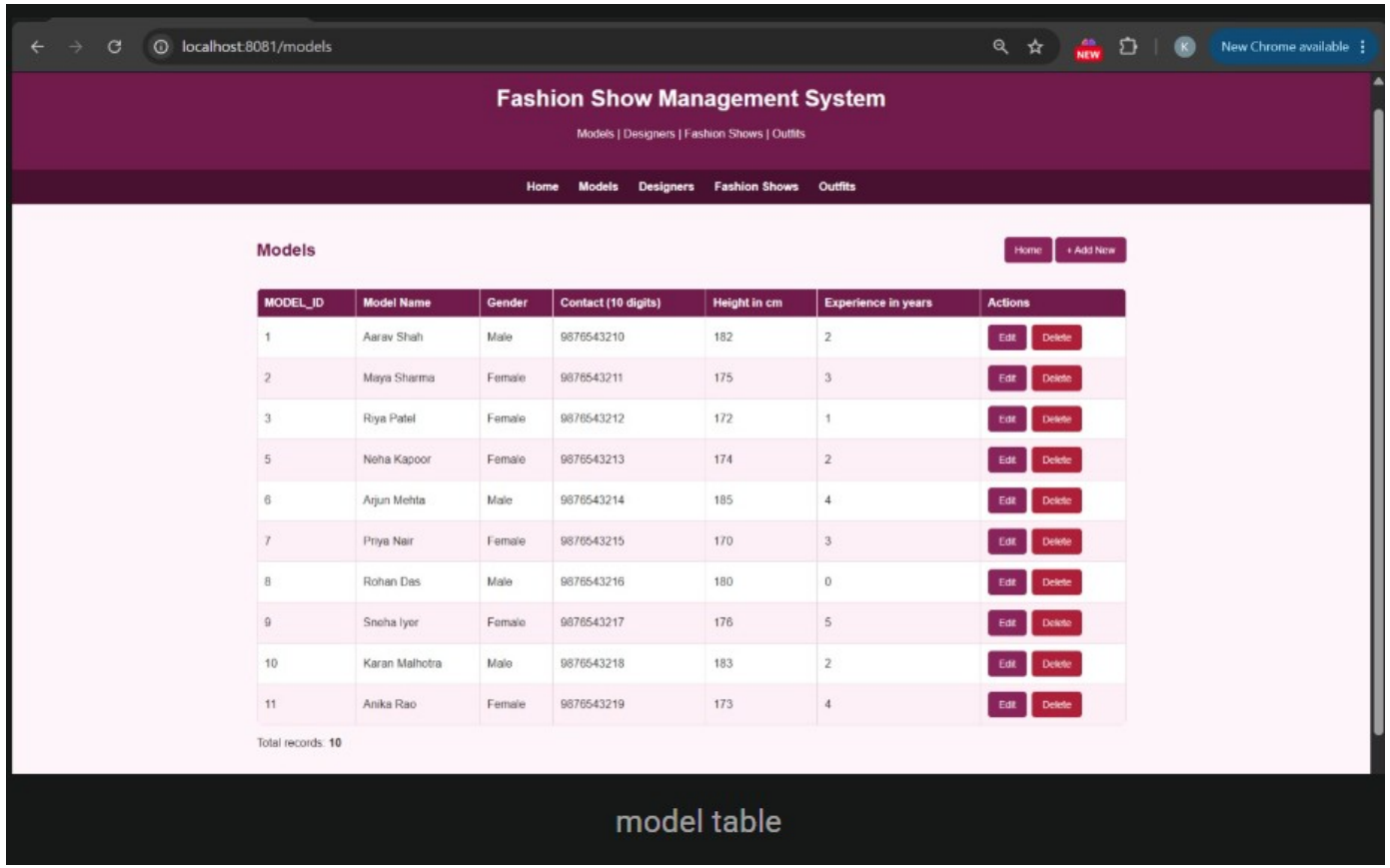
WHAT THIS SCREEN SHOWS

The second homepage capture shows the same navigation and module cards. It demonstrates the consistent layout used to move between the different sections.

DBMS / PROJECT EXPLANATION

A shared navigation bar makes the modules easy to reach from any page.

03 Models — Records Table



model table

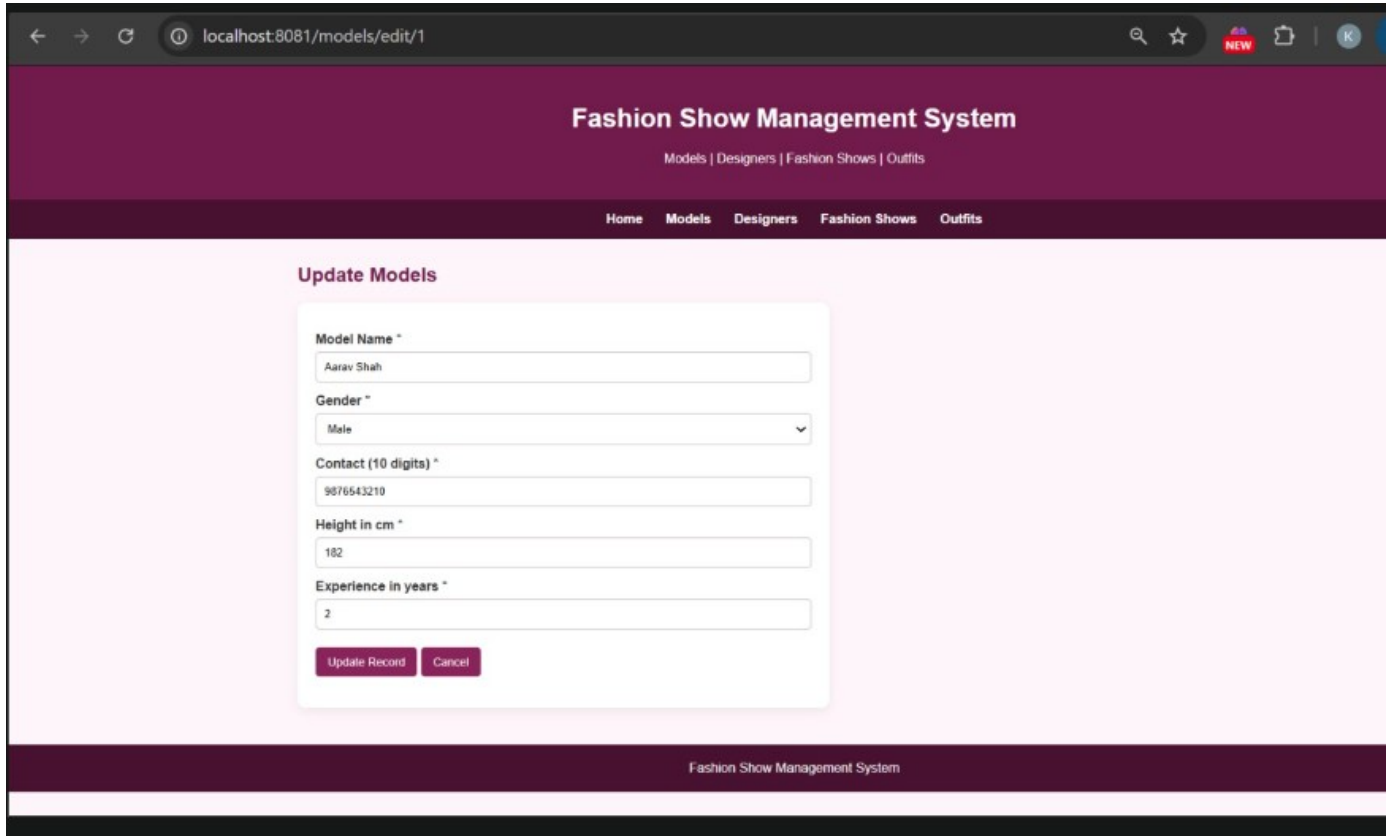
WHAT THIS SCREEN SHOWS

The Models table displays the model ID, name, gender, contact number, height and experience. Each record has Edit and Delete actions, and the page provides an Add New option.

DBMS / PROJECT EXPLANATION

READ: model records are displayed in a table. The model ID identifies each row for later updates or deletion.

04 Models — Edit Record



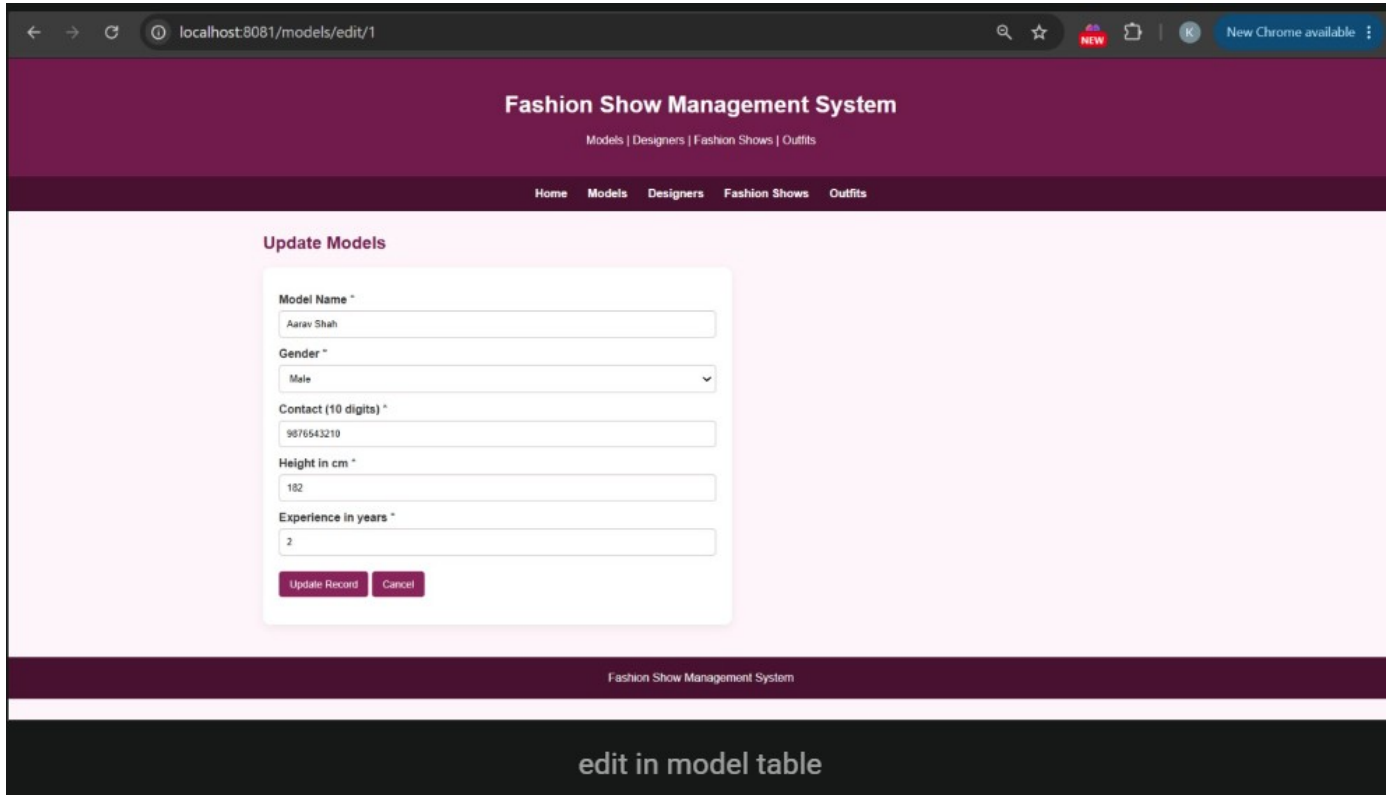
WHAT THIS SCREEN SHOWS

The edit form displays the selected model's current details. The user can change the name, gender, contact number, height and experience, then select Update Record.

DBMS / PROJECT EXPLANATION

UPDATE: the selected model ID determines which database record is changed. The application should validate the new values before saving.

05 Models — Update Form



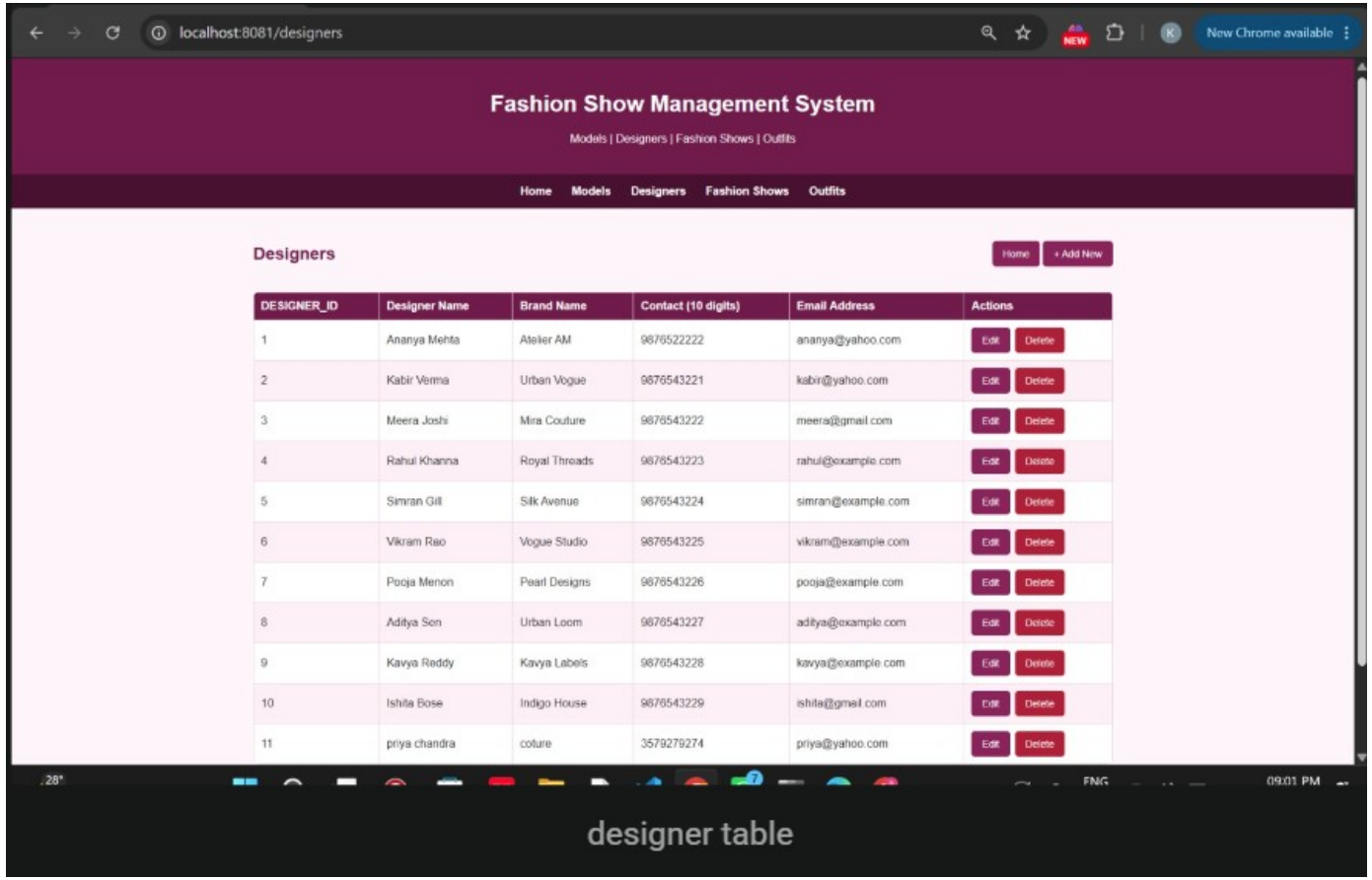
WHAT THIS SCREEN SHOWS

This capture shows the model form with existing values already filled in. The user can review the current information and change only the fields that need updating.

DBMS / PROJECT EXPLANATION

Pre-filled forms help prevent unnecessary re-entry and support accurate updates.

06 Designers — Records Table



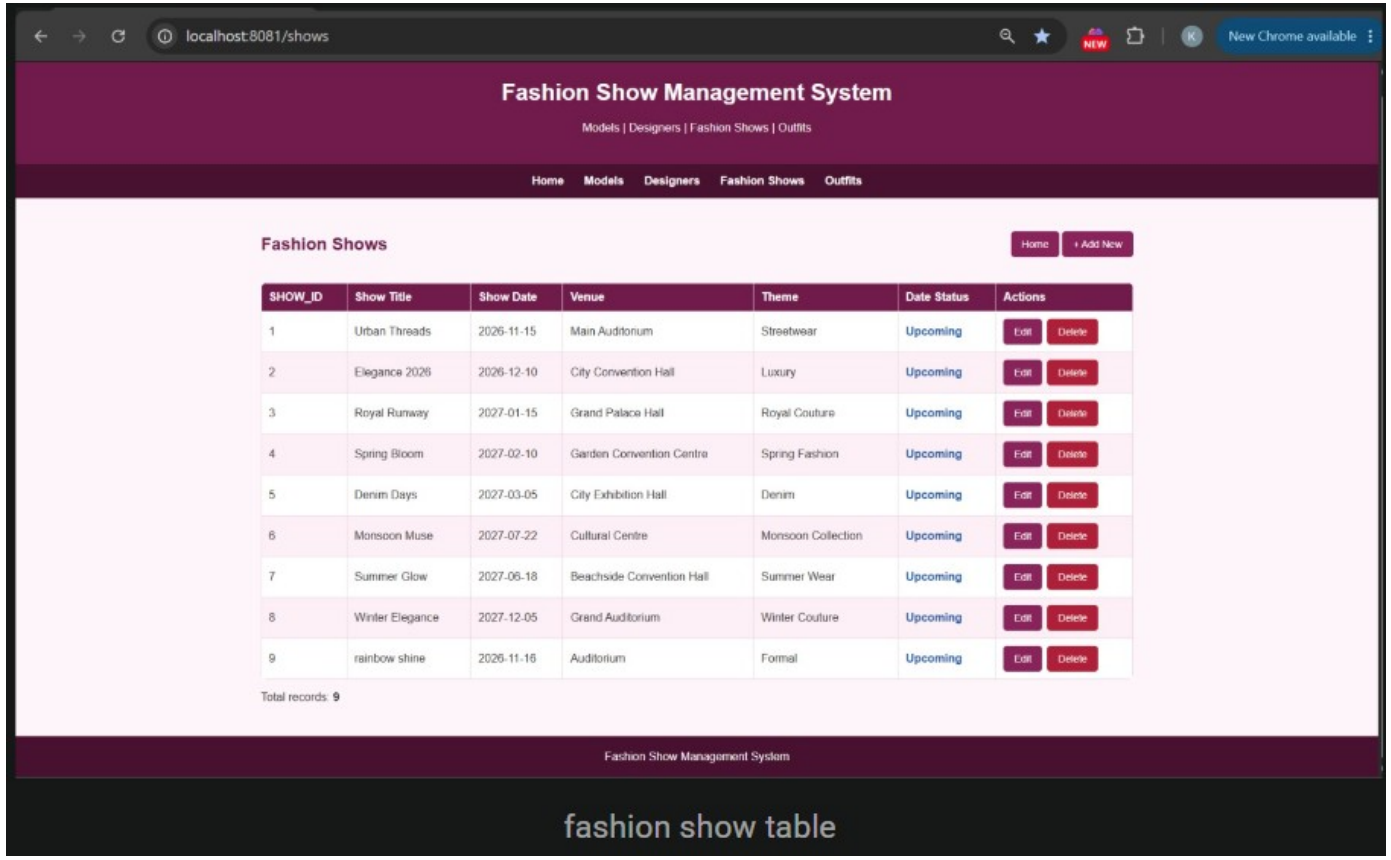
WHAT THIS SCREEN SHOWS

The Designers table lists designer ID, designer name, brand name, contact number and email address. Edit and Delete actions are available for each record.

DBMS / PROJECT EXPLANATION

Designer ID uniquely identifies a designer and is used to connect outfits to the correct designer.

07 Fashion Shows — Schedule



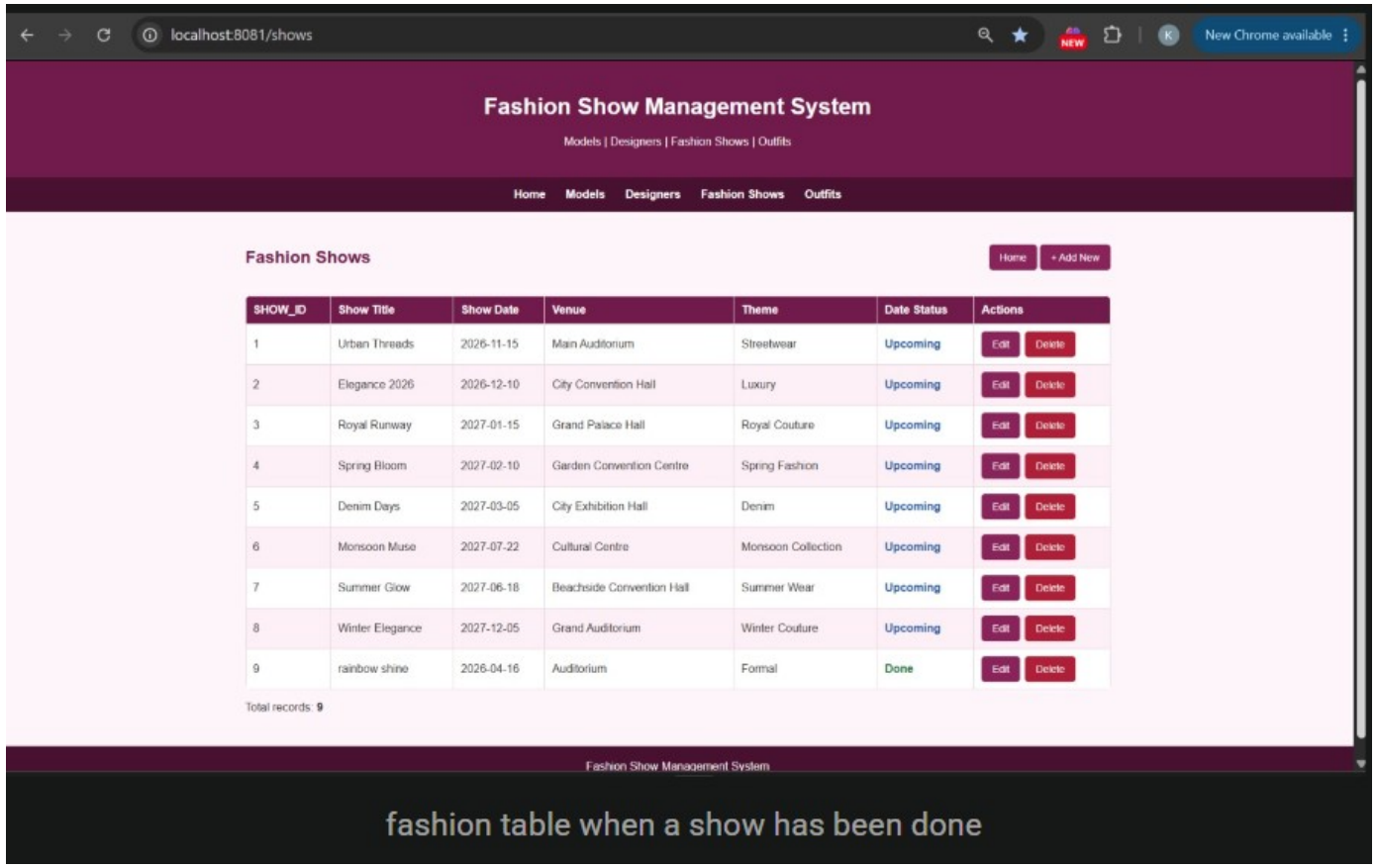
WHAT THIS SCREEN SHOWS

The Fashion Shows table displays the show title, date, venue, theme and date status. Users can add, edit or delete show records.

DBMS / PROJECT EXPLANATION

Show ID identifies each event. The Outfits table refers to a show through Fashion Show ID.

08 Fashion Shows — Completed Event



fashion table when a show has been done

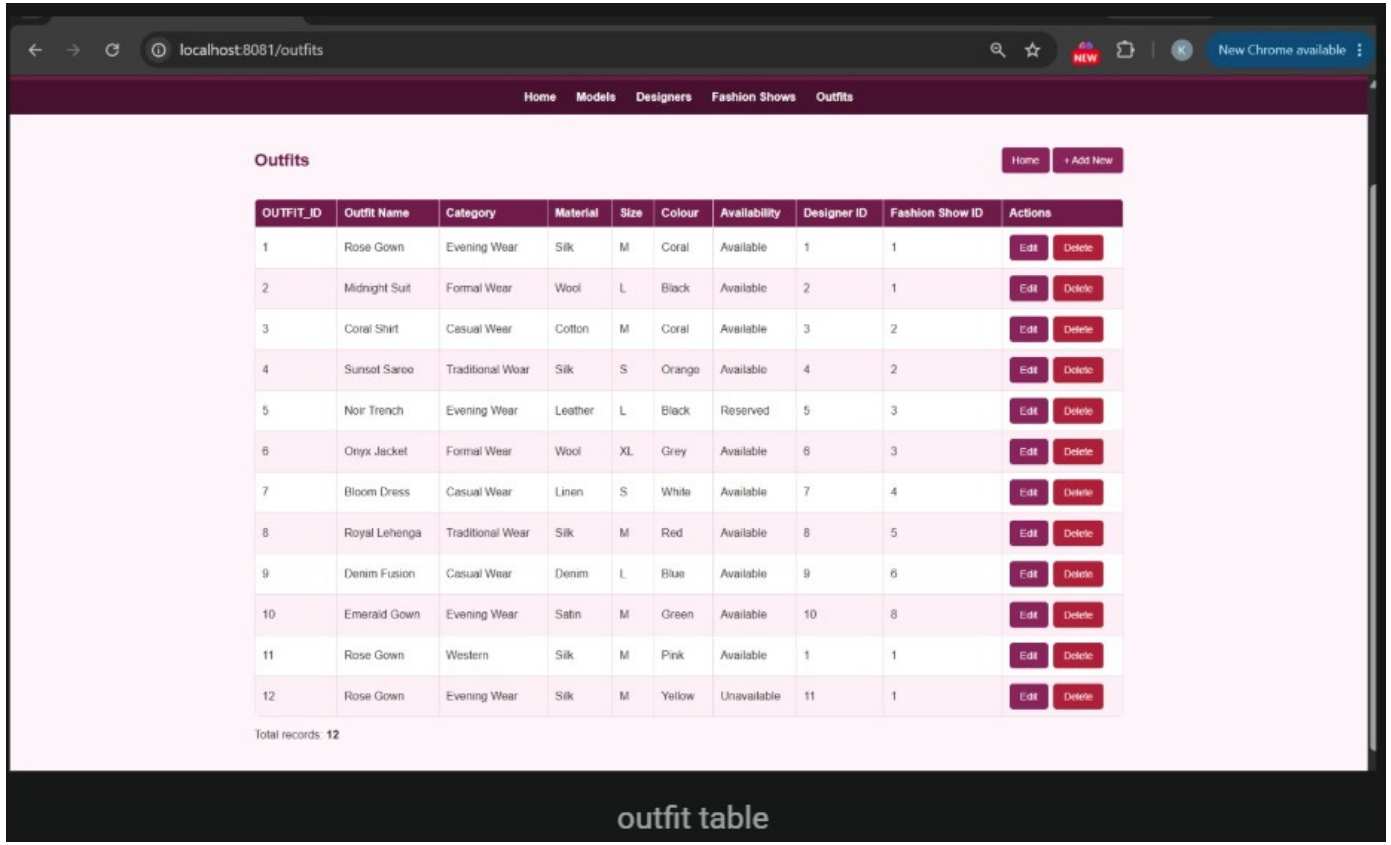
WHAT THIS SCREEN SHOWS

This screenshot shows an event marked Done while future events are shown as Upcoming. The status makes the event schedule easier to understand.

DBMS / PROJECT EXPLANATION

The displayed status distinguishes completed events from upcoming events.

09 Outfits — Records Table



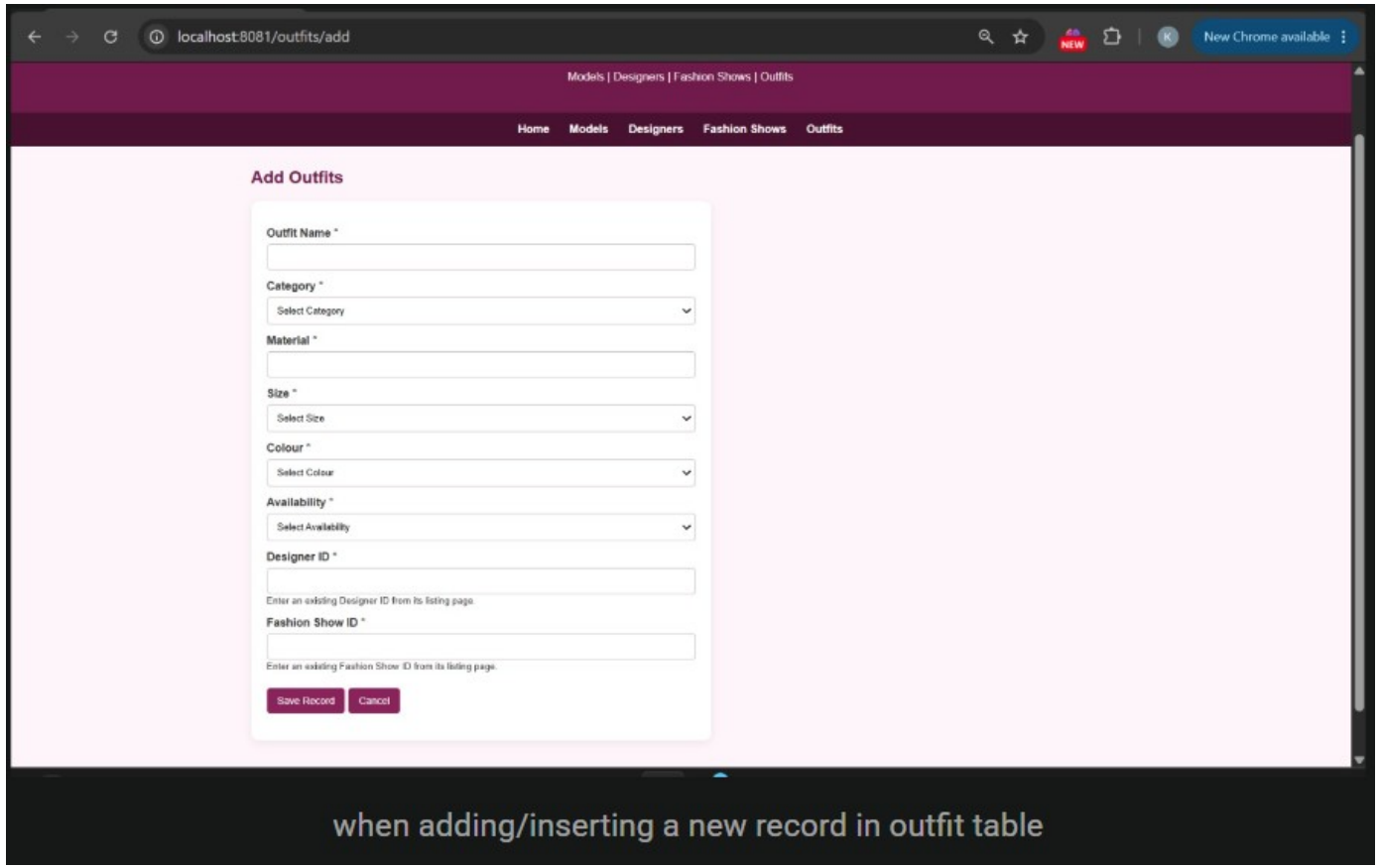
WHAT THIS SCREEN SHOWS

The Outfits table displays outfit name, category, material, size, colour and availability. It also shows Designer ID and Fashion Show ID.

DBMS / PROJECT EXPLANATION

These foreign-key values connect an outfit to its designer and fashion show.

10 Outfits — Add New Record



WHAT THIS SCREEN SHOWS

The Add Outfits form collects the outfit details and asks for Designer ID and Fashion Show ID so the new outfit can be linked to existing records.

DBMS / PROJECT EXPLANATION

CREATE: the form collects a new outfit record. The referenced designer and show IDs should exist before the record is saved.

THANK YOU

Swayamshree Saswatee Sahu • 25BAI0210

BACSE202 — Database Systems